

Contents

SQL CTE Validation Prompt Chain	1
Use Case	1
Required Local Preparation	1
Prompt Sequence	1
Related Reference Drawer	2
Relationship To Existing Flows	2
SQL CTE Validation Chain - 01 CTE Intake	2
SQL CTE Validation Chain - 02 CTE Logic Map	3
SQL CTE Validation Chain - 03 Stepwise Validation	3
SQL CTE Validation Chain - 04 Refactor Review	4
SQL CTE Validation Chain - 05 Final CTE Checklist	5

SQL CTE Validation Prompt Chain

Use Case

Use this chain when a human analyst needs Edge Copilot to validate, explain, or improve Oracle SQL built with common table expressions.

The workflow forces Copilot to map every CTE stage before recommending final SQL or rewrites. It is best for multi-stage queries where correctness depends on intermediate grain, joins, filters, derived fields, or dependency order.

Required Local Preparation

- Have the current Oracle SQL query ready, including the full **WITH** clause and final **SELECT**.
- Prepare compact schema context for source tables, key fields, join columns, date fields, status fields, and any known row-count expectations.
- Gather business rules, expected output grain, sample rows, and known edge cases.
- Keep validation evidence available, such as row counts by CTE, duplicate checks, null checks, reconciliation totals, or comparison results from the old and revised SQL.

Prompt Sequence

1. **01-cte-intake.md**: provide the SQL and context, then block rewrites until the CTE inventory is complete.
2. **02-cte-logic-map.md**: build the required CTE map covering purpose, dependencies, source tables, fields, grain, joins, filters, outputs, and checks.
3. **03-stepwise-validation.md**: turn the map into executable stage-level validation checks.
4. **04-refactor-review.md**: review possible rewrites only after the map and validation checks are complete.
5. **05-final-cte-checklist.md**: perform a final correctness, readability, and Oracle SQL safety review before accepting the query.

Related Reference Drawer

- `references/oracle-sql/sections/cte_patterns.md`

Use the drawer when detailed CTE guidance is needed, such as naming patterns, dependency ordering, materialization considerations, recursive CTE cautions, or stage-by-stage validation patterns. Do not duplicate long Oracle SQL tutorial content in the prompt response.

Relationship To Existing Flows

- Use `prompt-chains/sql-general/` first when the request is broad, simple, or not clearly centered on CTE validation.
- Use this chain after `sql-general` when the task becomes a multi-CTE correctness, explanation, or refactor review.
- Use `prompt-chains/query-tuning/` when the primary evidence is `EXPLAIN PLAN` output, runtime behavior, indexing, or optimizer diagnosis.
- Use this chain before `query-tuning` when the query must first be proven logically correct stage by stage.

SQL CTE Validation Chain - 01 CTE Intake

Act as an Oracle SQL reviewer focused on common table expression correctness.

This is a CTE validation workflow, not a rewrite workflow. Do not produce final SQL, replacement SQL, or optimized SQL yet.

First, inventory the query and identify what context is missing before a full CTE map can be completed.

Current Oracle SQL:

<paste the full Oracle SQL query here, including the WITH clause and final SELECT>

Business objective:

<paste the reporting goal, business rule, expected result, or acceptance criteria here>

Known schema context:

<paste source tables, columns, keys, join notes, row-count notes, date fields, status fields, >

Known validation evidence:

<paste row counts, sample rows, duplicate checks, reconciliation totals, or observed incorrect>

Return:

1. Overall query objective, as inferred.
2. Complete list of CTE names in dependency order.
3. Final output target and expected output grain, if inferable.
4. Source tables referenced by the query.
5. Business rules or field definitions that are unclear.
6. Schema, key, or sample-data context needed before mapping each CTE.
7. Specific clarification questions required before rewrite or final SQL work.

Do not generate a final rewrite until the CTE map is complete. Point me to [references/oracle-sql/sections/cte](#) if detailed CTE guidance is needed, but do not repeat long tutorial content from that drawer.

SQL CTE Validation Chain - 02 CTE Logic Map

Continue the Oracle SQL CTE validation workflow.

Using the query and context already provided, build a complete CTE map before any rewrite, optimization, or final SQL recommendation.

Current CTE inventory and clarifications:

<paste Copilot's CTE inventory and any answered clarification questions here>

Additional schema or business context:

<paste any new table definitions, field meanings, keys, sample rows, or business rules here>

Return one row per CTE using exactly this table shape:

cte	purpose	source tables	input fields	joins/filters	output grain	output fields	validation check
-----	---------	---------------	--------------	---------------	--------------	---------------	------------------

For each CTE, identify:

1. The CTE name and its purpose.
2. Dependencies on earlier CTEs.
3. Source tables and source CTEs used.
4. Input fields used from each source.
5. Joins and filters introduced at that stage.
6. Output row grain at that stage.
7. Output fields produced by that stage, including derived fields.
8. Validation check proving the stage behaves as intended.

After the table, return:

1. Dependency order from earliest source stage to final output.
2. Any missing source fields or unresolved aliases.
3. Grain changes between stages.
4. Filters or joins whose placement could change results.
5. CTEs that cannot yet be validated and why.

Do not generate a final rewrite until every CTE has a complete purpose, dependency, source-field, output-field, grain, join/filter, and validation-check entry. Use [references/oracle-sql/sections/cte_patterns](#) only when detailed CTE guidance is needed.

SQL CTE Validation Chain - 03 Stepwise Validation

Continue the Oracle SQL CTE validation workflow.

Use the completed CTE map to design stage-by-stage validation checks. Do not rewrite the query yet.

Completed CTE map:

<paste the completed CTE map table here>

Available validation evidence:

<paste row counts, sample rows, expected totals, known edge cases, or old-vs-new comparison ev

Return:

1. Validation sequence in CTE dependency order.
2. A row-count check for each CTE.
3. A grain or duplicate check for each CTE.
4. A null, unmatched join, or dropped-row check where relevant.
5. A filter-placement check where a CTE introduces filters.
6. A derived-field check where a CTE creates calculated columns.
7. A reconciliation check between the final output and the business objective.
8. Expected pass/fail signal for each check.

For any check that needs SQL, provide small Oracle SQL snippets that inspect intermediate stages. Keep snippets focused on validation, not final rewrites.

Flag any CTE that still lacks enough information to validate safely. Do not generate a final rewrite until the validation sequence covers every CTE.

SQL CTE Validation Chain - 04 Refactor Review

Continue the Oracle SQL CTE validation workflow.

Only review refactor or rewrite options if the CTE map is complete and the stagewise validation checks have been defined.

Completed CTE map:

<paste the completed CTE map table here>

Stepwise validation plan and results:

<paste validation checks and any observed results here>

Refactor goal:

<paste the goal: readability, safer CTE names, removing repeated logic, reducing grain risk, p

Return:

1. Whether the query is ready for refactor review.
2. Any CTEs that must not be changed until more validation evidence exists.
3. Safe readability improvements, such as clearer CTE names, alias cleanup, or separating mixed-grain logic.
4. Logic-preserving rewrite candidates, if safe.
5. For each candidate, the CTE stages affected and why behavior should remain equivalent.
6. Validation checks that must be rerun after each candidate.
7. Risks that could change row grain, filter scope, join behavior, or output fields.

If the CTE map is incomplete, stop and list the missing map fields instead of producing rewritten SQL. If detailed CTE guidance is needed, point to `references/oracle-sql/sections/cte_patterns.md` without duplicating long tutorial content.

SQL CTE Validation Chain - 05 Final CTE Checklist

Continue the Oracle SQL CTE validation workflow.

Use this final checklist before accepting a CTE-based Oracle SQL query or rewrite.

Final or proposed Oracle SQL:

<paste **the** final **or** proposed Oracle SQL here>

Completed CTE map:

<paste the completed CTE map table here>

Validation results:

<paste row counts, duplicate checks, reconciliation totals, sample-output checks, or old-vs-new>

Return:

1. Final list of CTEs in dependency order.
2. Confirmation that every CTE has a stated purpose.
3. Confirmation that every CTE lists source tables or source CTEs.
4. Confirmation that every CTE lists input fields and output fields.
5. Confirmation that every CTE has an output grain.
6. Confirmation that joins and filters introduced at each stage are documented.
7. Confirmation that each CTE has a validation check and observed result.
8. Any remaining correctness risks.
9. Any remaining readability or maintainability risks.
10. Whether the final SQL can be accepted as validated.

If any CTE is missing purpose, dependencies, source tables, input fields, joins/filters, output grain, output fields, or validation evidence, do not approve the final SQL. List the missing items and the next check to run.